



Graphic Era (Deemed to be University), Dehradun

Graphic Era University, Dehradun
Department of Computer Science & Engineering
Project Based Learning (PBL) - Phase I Synopsis

DYNAMIC SMART CITY ENGINE

A Dynamic Incident Prioritization and Resource Coordination System for Urban Services

Course: Data Structure and Programming in C++ | Semester: 3rd

Team ID	DSCPP-III-2026-T374	Semester	3rd
Course	Data Structure and Programming	ng in C++Mentor	Mr. Kireet Joshi
Name	Roll No.	Section	Role
Pavani Jain	260222072	DS1	Team Leader
Karan Babbar	2029058	SEC-B	Team Member
Manan Kumar	2028450	DS2	Team Member

Mentor email: kireetjoshi@geu.ac.in

1. Motivation

Our team selected this project after discussing a practical problem. If several city-related complaints come at the same time, it is not enough to only store them. The system should also help decide which problem needs attention first. Smart cities may collect information through cameras, sensors, GPS and other sources, but someone or some system still has to decide what should happen next.

We used Dehradun as the background for our idea because we study here and the city is part of the Smart City Mission. We also looked at material related to the Smart City project and CAG observations. This helped us understand that having technology available is not the same as using it effectively.

For example, a broken traffic signal at a busy crossing and a routine complaint may both be reported at the same time. The two cases should be recorded, but the response may not be equally urgent. Seriousness, number of people affected, location, available resources and waiting time can influence the decision. This is the type of situation our project will handle.

The Dynamic Smart City Engine will be a C++ prototype. It will use sample incidents instead of live city data. The program will keep incidents organised, calculate a priority, check the available resources and suggest what can be handled next. An incident can then move through different stages until it is closed.

2. State of the Art / Current Solution

Smart City systems can receive information from different sources such as cameras, sensors, GPS and traffic devices. This information may be shown through a central monitoring or command-and-control system.

For our project, we are mainly interested in what happens after the information is received. In many situations, a person may still have to decide which incident is urgent, which resource is free and what action should be taken. Our prototype focuses on this decision-making part.

The proposed engine will store incident details and apply simple rules. It will compare incidents according to priority and match them with available resources. A graph will represent selected city locations for routerelated calculations. The project is not meant to replace an actual Smart City platform. It is a small working model that demonstrates the required course concepts in one application.

3. Project Goals and Milestones

The main goal is to build a C++ prototype that can handle city incidents dynamically. Another goal is to show the use of Data Structures and Object-Oriented Programming in a single project. **Main Goals**

- Store different categories of city incidents.
- Assign priority using selected factors.
- Keep information about selected infrastructure conditions.
- Maintain available resources and suggest a suitable allocation.
- Represent selected locations using a graph.
- Find a route between a resource and an incident location.
- Track the status of an incident until it is resolved.
- Use Data Structures and C++ OOP concepts in separate modules.

Initial Milestones

- Finalize the problem and project requirements.
- Design the classes and select suitable data structures.
- Create the basic incident module.
- Add priority calculation.
- Add resource information and allocation.
- Build the graph and route module.
- Implement searching, sorting, hashing and tree-based organisation.
- Add file handling and saved reports.
- Test the program with sample cases based on possible city situations.

4. Project Approach

The first version will be a C++ simulation. We will not model the complete city. Only selected zones and locations will be used. An incident record can include the incident type, location, severity, number of people affected, reporting time and current status.

Different data structures will be used for different jobs. Linked lists can store incident records, while a queue can hold incidents waiting for processing. A graph will represent the selected city network. Hashing can help locate records using an ID. Trees can organise data for searching, and sorting can place incidents in priority order.

For OOP, we plan to use a common abstract class named City Event. Classes such as TrafficEvent, WasteEvent, Infrastructure Event and Emergency Event can inherit from it. Each type can calculate its priority in its own way through polymorphism. This will keep the program more organised than putting every case inside one large conditional block.

The program will use sample input. This keeps the first version realistic for a student project because access to live Smart City databases or sensor feeds is outside the present scope. Different sample cases can still be used to test urgent, normal and delayed incidents.

5. System Architecture

Simulated City Data / User Input

↓

Incident Collection Module

↓

Incident Management Engine

(Priority Calculation • Infrastructure Status Check • Resource Availability Check • City Network / Route Calculation)

↓

Recommended Action & Resource Assignment

↓

Status Tracking and Report Generation

Main Components

- Incident Management Module
- Priority Engine
- Infrastructure Monitor
- Resource Management Module
- City Graph and Route Module
- Data Storage and File Module
- Report / Output Module

6. Project Outcome / Deliverables

The final program will be a C++ prototype called Dynamic Smart City Engine. It will work with sample city incidents. The program will give every incident a priority based on the rules used in the project. Factors such as seriousness, impact and waiting time can affect the result.

The user will be able to add an incident and search for an existing record. Incidents can be viewed in priority order and their status can be changed as the situation moves forward. The program will also keep details about resources and selected infrastructure. A small graph will be used for the location and route part.

The project brings several syllabus topics together in one practical application. The planned data structures include linked lists, queues, stacks, trees, graphs and hashing.

Searching, sorting will also be used.

C++ - project will use class, inheritance, polymorphism, abstract class, virtual function, test cases and presentation.

ASSUMPTION

This project is prototype

This project is displaying every scenario and routes for demonstration purpose only and are not connected to a live government system or real sensor network.

These assumptions are kept within the available time and scope of the course